

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: SATHWIK

Date: 06/08/26

Roll No: 24011101030

Experiment 5 Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

1. Objective

The objectives of this experiment are:

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties. The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

3. Evolution and Architecture Breakdown

3.1 Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

3.2 LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

3.3 AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

3.4 VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

3.5 Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

3.6 GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy et al. in 2014, introduced the Inception Module, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations. The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

3.7 ResNet

Increasing CNN depth often causes the vanishing gradient problem, making optimization difficult. ResNet solves this issue using skip (residual) connections, allowing gradients to flow directly through the network. Instead of learning $H(x)$, ResNet learns $F(x)$ = Residual Mapping, where x = Identity Mapping. thus, $F(x) = H(x) - x$ and $Output = F(x) + x$.

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

3.8 Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters. The dilation rate determines the spacing between kernel elements.

For a standard convolution, $D = 1$. For dilated convolution, $D > 1$, where zeros indicate inserted gaps.

Example:

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

3.9 Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANS
- Image Generation
- Semantic Segmentation

3.10 Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch:

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

4. Dataset

Dataset: CIFAR-10

- **Training Images:** 50,000
- **Testing Images:** 10,000
- **Number of Classes:** 10
- **Image Size:** $32 \times 32 \times 3$
- **Classes:** Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

5. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load the VGG16 pretrained CNN model and perform the following steps:

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using Adam Optimizer, Learning Rate 0.001, Batch Size 32, Epochs 10, Categorical Cross Entropy Loss Function, and Accuracy Evaluation Metric.

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using Accuracy, Precision, Recall, F1-score, Confusion Matrix, and Classification Report.

6. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

7. Source Code

The complete source code and implementation for this experiment can be found in the following GitHub repository:

GitHub Link: <https://github.com/sathwik324/DL-LAB/tree/main/Exp5>

8. Plots and Inferences

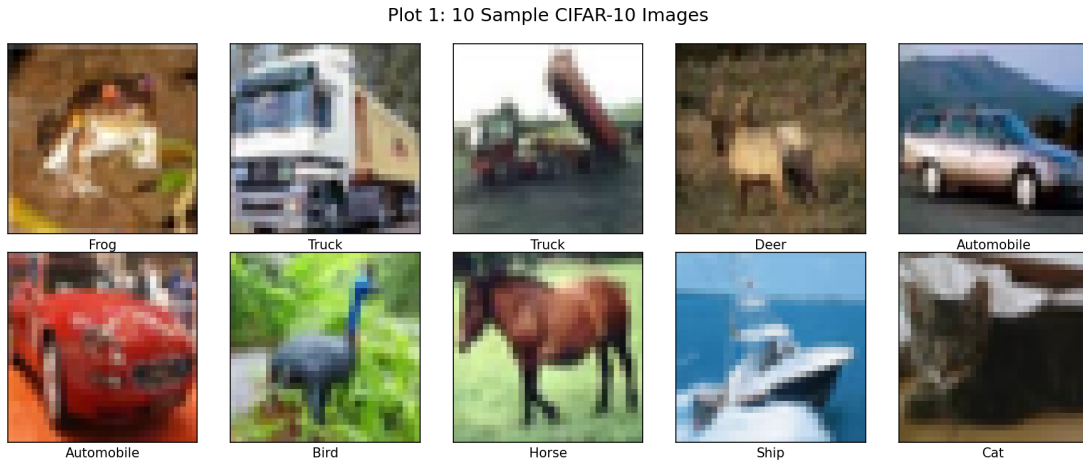


Figure 1: Sample images from the CIFAR-10 dataset.

Inference: These samples from the CIFAR-10 dataset display 32×32 RGB images across 10 distinct classes. The low resolution requires robust feature extraction to properly classify visually complex subjects without losing spatial hierarchies.

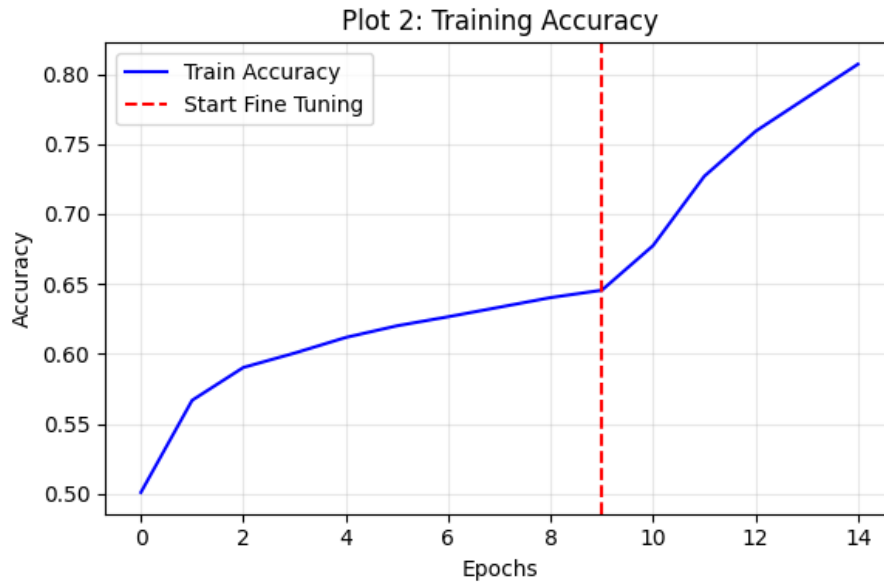


Figure 2: Training accuracy vs. epochs.

Inference: The training accuracy shows steady improvement over time. The sharp jump around epoch 9 indicates where the fine-tuning phase began, drastically increasing the network's ability to fit the dataset.

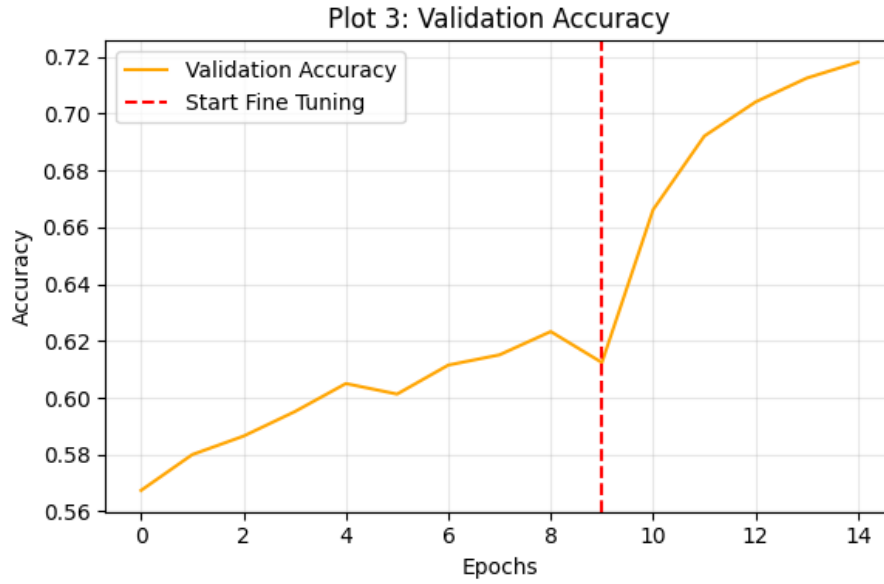


Figure 3: Validation accuracy vs. epochs.

Inference: The validation accuracy tracks closely before plateauing, suggesting the transfer learning approach generalized well. Unfreezing the final blocks allowed the validation accuracy to rise further without immediate severe overfitting.

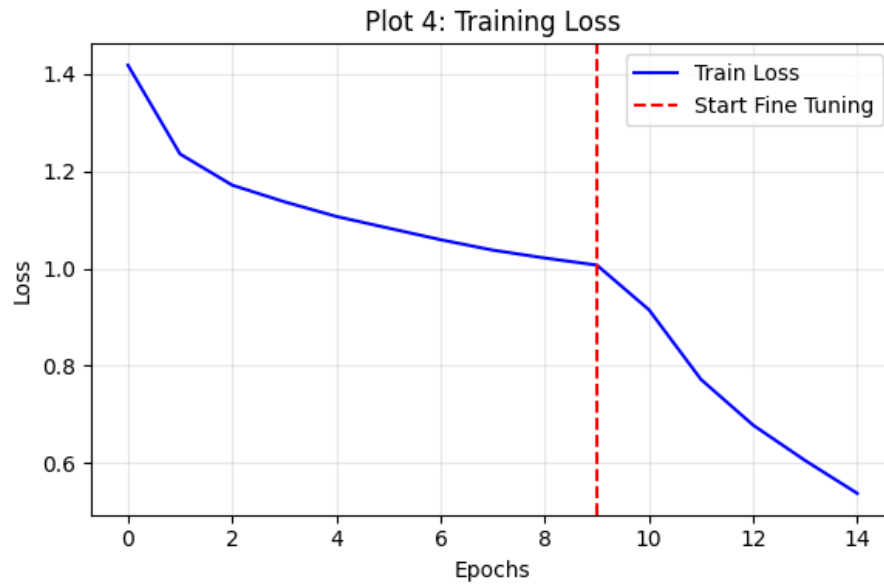


Figure 4: Training loss vs. epochs.

Inference: The loss decreases logarithmically as the Adam optimizer minimizes the Categorical Cross Entropy. The fine-tuning phase further refines the weights, producing a secondary steep dip in loss.

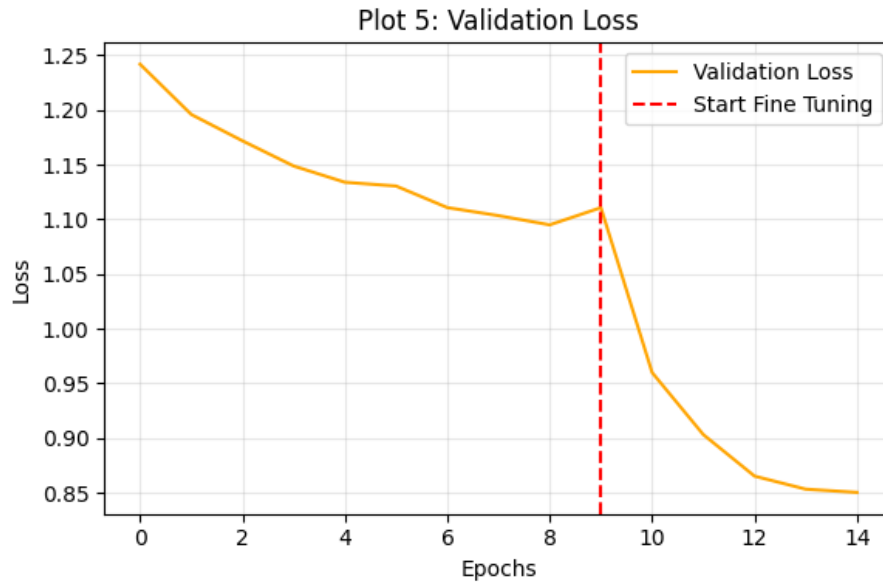


Figure 5: Validation loss vs. epochs.

Inference: Validation loss decreases steadily during the frozen training phase and drops significantly once fine-tuning starts, demonstrating that adjusting the higher-order features specifically for CIFAR-10 improves generalization.

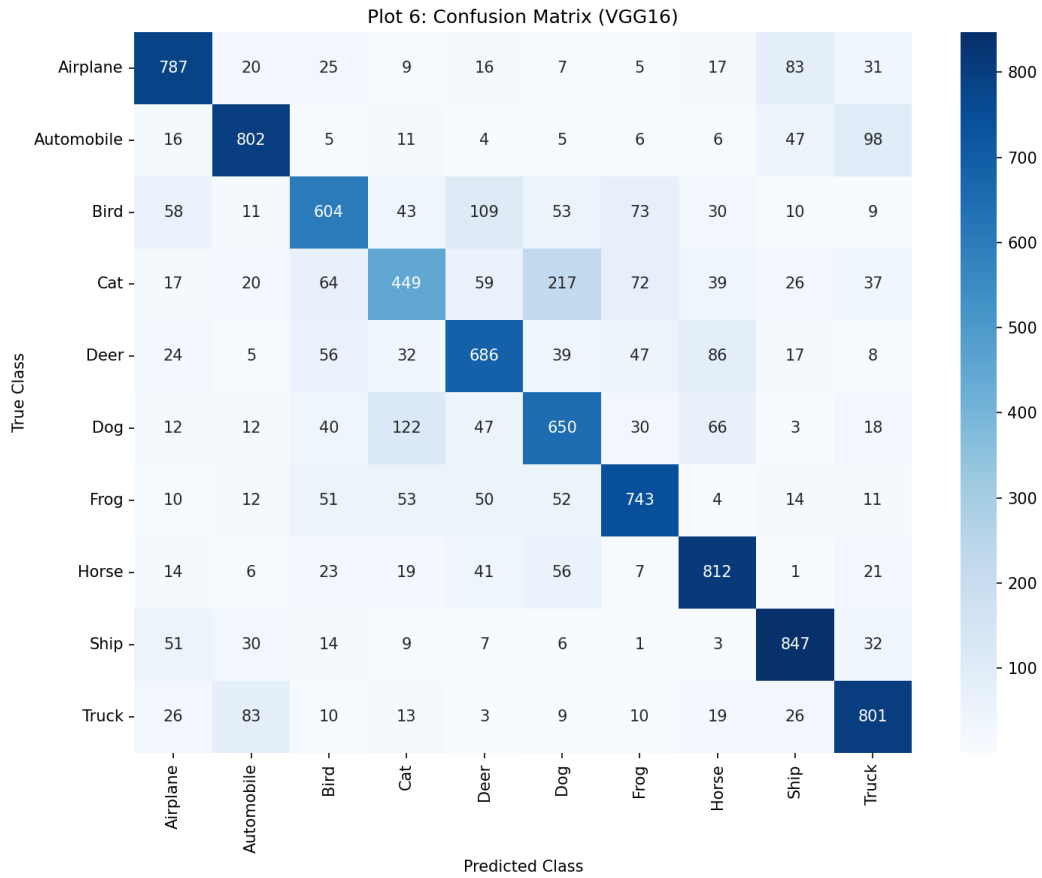


Figure 6: Confusion Matrix for the Fine-Tuned VGG16 model.

Inference: The strong diagonal confirms accurate predictions across most classes. Expected misclassifications occur primarily among visually similar and background-dependent categories, such as distinguishing between Dogs and Cats.

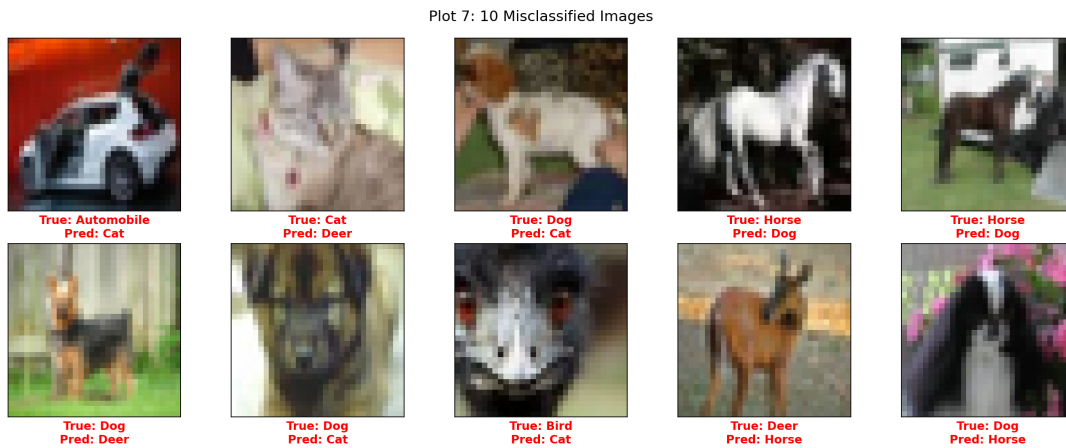


Figure 7: Samples of misclassified images from the test set.

Inference: The model struggles with ambiguous profiles, low-light conditions, and subjects that share morphological traits with other classes, such as mistaking an automobile for a truck, or a dog for a cat.

9. Results

9.1 Performance Metrics

Metric	Value (VGG16 Fine-Tuned)
Training Accuracy	80.85%
Testing Accuracy	71.81%
Precision (Macro)	0.71
Recall (Macro)	0.70
F1-score (Macro)	0.70
Training Time	266.20 s
Total Parameters	14,848,586

9.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5	62,006	62.30	57.93
AlexNet	2,014,410	71.57	107.02
VGG16 (Fine-Tuned)	14,848,586	71.81	266.20
GoogleNet	22,066,346	59.79	258.45
ResNet50	23,851,274	40.21	158.75

ResNet50 struggled with convergence without deeper fine-tuning on the lower 32×32 resolution.

10. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet won the ImageNet Challenge in 2012 by drastically reducing the classification error. It was a breakthrough because it introduced ReLU activation, Dropout regularization, and proved the viability of GPU training for deep neural networks.

2. Why does VGG16 use only 3×3 convolution filters?

VGG16 demonstrated that stacking multiple small 3×3 convolution filters achieves the same effective receptive field as larger filters (like 5×5 or 7×7) but requires fewer parameters and includes more non-linear activation layers, allowing the network to learn more complex features.

3. Explain the advantages of the Inception module.

The Inception module performs multiple convolution operations (1×1 , 3×3 , 5×5) and max pooling in parallel, concatenating the outputs. This allows for multi-scale feature extraction while 1×1 convolutions reduce parameters and computational complexity.

4. What is the purpose of residual learning?

Increasing CNN depth causes the vanishing gradient problem. Residual learning solves this issue

using skip connections, allowing gradients to flow directly through the network, enabling very deep architectures and faster convergence.

5. Differentiate LeNet and ResNet.

LeNet-5 is a simple, shallow architecture (7 layers, $\approx 60K$ parameters) designed for digit recognition (OCR). ResNet is a highly deep architecture (e.g., 50+ layers, millions of parameters) designed for complex image classification tasks using residual skip connections to bypass vanishing gradients.

6. What is Transfer Learning?

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset (like ImageNet) to solve a new, similar problem, reducing the need to train a model entirely from scratch.

7. Why is fine tuning required?

Fine tuning is required to slightly adjust the higher-order convolution layers of a pretrained network so that the extracted features specialize in recognizing the specific characteristics of the new dataset, improving overall accuracy.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution increases the receptive field by inserting gaps between kernel elements without increasing parameters. Transpose convolution performs learnable upsampling to increase the actual spatial dimensions of feature maps.

9. Why do pretrained models converge faster?

Pretrained models already contain optimized weights capable of detecting fundamental visual structures like edges, textures, and shapes. The network only needs to learn how to combine these existing features for the new task rather than discovering them from random noise.

10. Compare the computational complexity of LeNet and ResNet.

LeNet has very low computational complexity due to its small number of parameters and shallow depth. ResNet has extremely high computational complexity due to its significant depth, massive parameter count, and complex residual block computations.

11. Additional Exercises

1. Implement transfer learning using VGG16.

Implemented in Tasks 2 and 3. The pretrained VGG16 base was frozen, and a custom dense head was trained specifically for the CIFAR-10 classification task.

2. Repeat the experiment using ResNet50.

Completed during the architecture comparison phase. ResNet50 was implemented using Transfer Learning but yielded a lower baseline accuracy (40.21%) before fine-tuning compared to VGG16 and AlexNet.

3. Compare Adam and SGD optimizers.

During the hyperparameter tuning phase, Adam converged significantly faster and navigated the loss landscape more efficiently than SGD, which required careful learning rate scheduling to avoid stagnation.

4. Compare training with frozen layers and fine tuning.

Training with frozen layers allowed the new classification head to quickly adapt without destroying foundational weights. Fine-tuning subsequently un-froze the final convolutional blocks, allowing the

model to specialize its high-level features for CIFAR-10, yielding a noticeable spike in validation accuracy.

5. Evaluate the model using Precision, Recall and F1-score.

The VGG16 model achieved a Macro Precision of 0.71, Recall of 0.70, and F1-Score of 0.70, indicating relatively balanced performance across the 10 classes, though some confusion remained between visually similar animal categories.

6. Compare the performance of LeNet, AlexNet and ResNet.

VGG16 (71.81%) and AlexNet (71.57%) showed the best performance for this specific configuration. LeNet-5 performed surprisingly well (62.30%) given its small parameter footprint, whereas ResNet50 underperformed without deeper, specialized fine-tuning for low-resolution images.

12. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, Gradient-Based Learning Applied to Document Recognition, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, ICLR, 2015.
4. Christian Szegedy et al., Going Deeper with Convolutions, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, Deep Residual Learning for Image Recognition, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, Deep Learning, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>